

Semana 7  
ISIS-1611: Inteligencia Artificial

Carla González

22 de julio de 2026

## 1. Inferencia en LPO: Cierre

Esta clase cierra la unidad de lógica e inferencia (AIMA Cap. 7–9) antes de pasar a planeación: se repasan model checking en lógica proposicional, encadenamiento hacia adelante y hacia atrás sobre cláusulas de Horn, unificación y el occurs check, y resolución por refutación en LPO. Luego se aplica todo el repaso a una guía de ejercicios completa, resuelta paso a paso.

### 1.1. Lógica Proposicional: Entailment y Model Checking

**Definición 1.1** (Implicación lógica).  $KB \models \alpha$  (“ $KB$  implica  $\alpha$ ”) si y solo si  $\alpha$  es verdadera en *todo* modelo en el que  $KB$  es verdadera. Equivalentemente (teorema de refutación):

$$KB \models \alpha \quad \text{si y solo si} \quad KB \wedge \neg\alpha \text{ es insatisfacible.}$$

**Model checking (TT-Entails?)**: para un número finito de símbolos proposicionales, se enumeran las  $2^n$  interpretaciones posibles (tabla de verdad completa), se marcan los **modelos** de  $KB$  (filas donde  $KB$  es verdadera) y se verifica si  $\alpha$  es verdadera en todos ellos. Un solo **contramodelo** (modelo de  $KB$  donde  $\alpha$  es falsa) basta para refutar  $KB \models \alpha$ . Es importante distinguir dos nociones que se confunden con frecuencia:

- $\alpha$  es **consistente** con  $KB$ : existe *al menos un* modelo de  $KB$  donde  $\alpha$  es verdadera ( $KB \wedge \alpha$  es satisfacible).
- $KB \models \alpha$ :  $\alpha$  es verdadera en **todos** los modelos de  $KB$ .

La primera es mucho más débil que la segunda.

### 1.2. Cláusulas de Horn y Encadenamiento hacia Adelante

**Definición 1.2** (Cláusula de Horn y cláusula definida). Una **cláusula de Horn** es una disyunción de literales con *a lo sumo un literal positivo* (puede no tener ninguno, en cuyo caso es una **cláusula meta** o de restricción, útil para expresar una contradicción). Una **cláusula definida** es una cláusula de Horn con *exactamente un* literal positivo, y se escribe como implicación

$$P_1 \wedge \cdots \wedge P_n \Rightarrow Q$$

(las premisas  $P_i$  deben ser todas verdaderas para concluir  $Q$ ). Todo el material de esta guía (Ejercicios 2 y 3) usa cláusulas *definidas*: por eso el encadenamiento hacia adelante y hacia atrás son aplicables y completos.

El **encadenamiento hacia adelante** (forward chaining) parte de los hechos conocidos y aplica reglas cuyas premisas ya están satisfechas, derivando nuevos hechos hasta alcanzar la meta o agotar la agenda.

**Definición 1.3** (PL-FC-ENTAILS?, con agenda y contadores). Se mantiene, para cada regla, un **contador** con el número de premisas aún no confirmadas, y una **agenda** FIFO de símbolos conocidos verdaderos que faltan por procesar. En cada paso se extrae el siguiente símbolo  $p$  de la agenda; si  $p$  es la meta, se retorna verdadero; si no, se marca  $p$  como inferido y, por cada regla donde  $p$  aparece en la premisa, se decrementa su contador; si un contador llega a 0, la conclusión de esa regla se agrega a la agenda.

**Algorithm 1** PL-FC-ENTAILS?( $KB, q$ ) — forward chaining con agenda y contadores

---

```

1:  $count[c] \leftarrow$  número de símbolos en la premisa de  $c$ , para cada cláusula  $c \in KB$ 
2:  $inferred[s] \leftarrow$  falso, para cada símbolo  $s$ 
3:  $agenda \leftarrow$  símbolos conocidos verdaderos en  $KB$ 
4: while  $agenda$  no está vacía do
5:    $p \leftarrow \text{POP}(agenda)$ 
6:   if  $p = q$  then return verdadero
7:   end if
8:   if  $inferred[p] =$  falso then
9:      $inferred[p] \leftarrow$  verdadero
10:    for cada cláusula  $c$  tal que  $p$  aparece en  $c.PREMISA$  do
11:      decrementar  $count[c]$ 
12:      if  $count[c] = 0$  then
13:        agregar  $c.CONCLUSION$  a  $agenda$ 
14:      end if
15:    end for
16:  end if
17: end while
18: return falso

```

---

- **Completo** para bases de conocimiento proposicionales de cláusulas de Horn (deriva exactamente los hechos implicados, ni más ni menos).
- **Dirigido por los datos** (*data-driven*): no usa la meta para decidir qué reglas disparar, así que puede derivar hechos verdaderos pero **irrelevantes** para la consulta.

### 1.3. Encadenamiento hacia Atrás

El **encadenamiento hacia atrás** (backward chaining) parte de la meta y busca reglas cuya conclusión coincida con ella; cada premisa de esas reglas se convierte, recursivamente, en una nueva **submeta**. Un hecho de la base resuelve una submeta directamente (hoja del árbol de prueba).

- **Dirigido por la meta** (*goal-driven*): solo explora lo relevante para probar la consulta.
- Cuando una regla no logra probar una submeta (no hay hechos ni otra regla aplicable, o todas las alternativas fallan), ocurre **backtracking**: se descarta el intento y se prueba la siguiente regla cuya conclusión coincida con esa submeta.
- Completo para cláusulas de Horn; es la base operacional de Prolog. Sin control de ciclos puede entrar en bucles infinitos si una submeta depende (directa o indirectamente) de sí misma.

### 1.4. Sustitución, Unificación y Occurs Check

**Definición 1.4** (Unificador más general, UMG).  $\text{UNIFY}(p, q) = \theta$  tal que  $\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$ , y  $\theta$  es el **más general** posible: cualquier otro unificador  $\theta'$  se obtiene componiendo  $\theta$  con una sustitución adicional. La unificación procede posición a posición: variable con término (se liga la variable), constante con constante idéntica (coincide sin generar ligadura), constantes distintas o predicados/aridades distintas (falla).

**Definición 1.5** (Occurs check). Antes de ligar una variable  $x$  a un término  $t$ , el **occurs check** verifica que  $x$  no aparezca dentro de  $t$ . Si  $x$  ocurriera en  $t$ , la ligadura  $x/t$  generaría un término

**infinito** (p. ej.  $x = f(x) = f(f(x)) = \dots$ ), que no es un objeto válido en LPO. Muchas implementaciones de Prolog omiten el occurs check por eficiencia, arriesgando unificaciones incorrectas que un motor completo rechazaría.

### 1.5. Forma Clausal y Resolución por Refutación en LPO

**Definición 1.6** (Forma normal conjuntiva (FNC) en LPO — pasos de conversión). Toda sentencia de LPO se convierte a FNC (conjunción de cláusulas, cada una disyunción de literales) siguiendo, en orden, estos pasos:

1. **Eliminar**  $\Leftrightarrow$  y  $\Rightarrow$ :  $\alpha \Leftrightarrow \beta$  se reescribe  $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$ , y  $\alpha \Rightarrow \beta$  se reescribe  $\neg\alpha \vee \beta$ .
2. **Mover  $\neg$  hacia adentro** (leyes de De Morgan y de cuantificadores):  $\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$ ,  $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$ ,  $\neg\forall x \Phi \equiv \exists x \neg\Phi$ ,  $\neg\exists x \Phi \equiv \forall x \neg\Phi$ ,  $\neg\neg\alpha \equiv \alpha$ .
3. **Estandarizar variables**: renombrar variables ligadas para que cada cuantificador use un nombre distinto (evita colisiones al combinar cláusulas de sentencias distintas).
4. **Skolemizar** (ver definición siguiente): eliminar cada cuantificador existencial reemplazando su variable por una constante o función de Skolem.
5. **Eliminar los cuantificadores universales** restantes (quedan implícitos: toda variable libre en una cláusula se entiende universalmente cuantificada).
6. **Distribuir  $\vee$  sobre  $\wedge$**  hasta que la fórmula sea una conjunción de disyunciones de literales; cada conjunto separado por  $\wedge$  es una cláusula distinta.

**Definición 1.7** (Skolemización). Si una variable existencial  $\exists y$  **no** está dentro del alcance de ningún  $\forall$ , se reemplaza  $y$  por una **constante de Skolem** nueva (un objeto concreto, aunque desconocido, que testifica la existencia). Si  $\exists y$  **sí** está dentro del alcance de  $\forall x_1, \dots, \forall x_k$ , se reemplaza  $y$  por una **función de Skolem** nueva  $f(x_1, \dots, x_k)$  (el testigo puede depender de esos universales). Por ejemplo,  $\forall x \exists y \text{ Padre}(y, x)$  (“todo el mundo tiene un padre”) se skolemiza como  $\forall x \text{ Padre}(S(x), x)$ :  $S(x)$  denota *el* padre de  $x$ , que en general depende de  $x$ .

■ **Regla de resolución (LPO):**

$$\frac{\ell_1 \vee \dots \vee \ell_k \quad \neg m_1 \vee \dots \vee \neg m_j}{\text{SUBST}(\theta, \ell_1 \vee \dots \vee m_j)} \quad \theta = \text{UNIFY}(\ell_i, m_i)$$

- **Refutación**: para probar  $KB \models \alpha$ , se agrega  $\neg\alpha$  (en FNC) al conjunto de cláusulas de  $KB$  y se aplica resolución hasta derivar la **cláusula vacía**  $\square$  (contradicción) —lo que confirma que  $KB \wedge \neg\alpha$  es insatisfacible— o hasta no poder generar cláusulas nuevas (en cuyo caso  $KB \not\models \alpha$ ).
- La resolución en LPO es **sana y completa por refutación** (teorema de completitud de Gödel), pero solo **semi-decidible**: si  $KB \models \alpha$ , se garantiza encontrar  $\square$  en un número finito de pasos; si  $KB \not\models \alpha$ , el procedimiento puede no terminar nunca.

## 2. Ejercicios: Guía de Lógica e Inferencia (con solución paso a paso)

Esta sección resuelve, de principio a fin, la *Guía de Ejercicios: Lógica e Inferencia*.

### 2.1. Bloque 1: Preguntas Conceptuales (Verdadero / Falso)

- a. “Si una sentencia  $\alpha$  es satisfacible, entonces  $KB \models \alpha$  para cualquier base de conocimiento  $KB$  consistente con  $\alpha$ .”

**Falso.** Satisfacibilidad de  $\alpha$  solo dice que  $\alpha$  es verdadera en *algún* modelo; que  $KB$  sea *consistente* con  $\alpha$  solo dice que  $KB \wedge \alpha$  tiene algún modelo. Ninguna de las dos garantiza que  $\alpha$  sea verdadera en **todos** los modelos de  $KB$ , que es lo que exige  $\models$ .

**Contraejemplo:**  $KB = \{P\}$ ,  $\alpha = Q$ .  $\alpha$  es satisfacible, y  $KB$  es consistente con  $\alpha$  (el modelo  $P=T, Q=T$  satisface ambos), pero  $KB \not\models Q$  (el modelo  $P=T, Q=F$  también es modelo de  $KB$  y no de  $Q$ ).

- b. “El encadenamiento hacia adelante (forward chaining) es completo para bases de conocimiento proposicionales formadas por cláusulas de Horn.”

**Verdadero.** Para cláusulas de Horn (cada regla con a lo sumo un literal positivo), el algoritmo con agenda y contadores deriva exactamente todos los símbolos atómicos implicados por  $KB$ : es sano (solo deriva lo que se sigue lógicamente) y completo (deriva todo lo que se sigue).

- c. “ $KB \models \alpha$  si y solo si la sentencia  $(KB \wedge \neg\alpha)$  es insatisfacible.”

**Verdadero.** Es el **teorema de refutación**, base de todo método de prueba por contradicción (model checking negativo y resolución): si  $\alpha$  fuera falsa en algún modelo de  $KB$ , ese modelo satisface simultáneamente  $KB$  y  $\neg\alpha$ ; si ningún modelo satisface ambos a la vez, entonces  $\alpha$  es verdadera en todo modelo de  $KB$ .

- d. “Durante la unificación es válido sustituir un término por una constante de Skolem, de la misma forma en que se sustituye una variable.”

**Falso.** Las constantes (y funciones) de Skolem se introducen *antes* de la unificación, durante la conversión a FNC, para eliminar cuantificadores existenciales; representan un objeto **fijo pero arbitrario** del dominio, no un comodín instanciable. La unificación solo liga **variables** (símbolos que aún pueden tomar cualquier valor) a términos; una constante de Skolem se comporta como cualquier otra constante: puede aparecer *dentro* de un término al que se liga una variable, pero ella misma no puede recibir una ligadura ni ser tratada como si fuera libre de unificarse con otro término distinto.

### 2.2. Ejercicio 1: El apagón en Paloquemao

Símbolos:  $P$  (falló la planta),  $S$  (sobrecarga en neveras),  $T$  (transformador dañado).

$$R1 : P \vee S \vee T \quad R2 : \neg(P \wedge S) \quad R3 : T \Rightarrow P \quad KB = R1 \wedge R2 \wedge R3$$

| $P$ | $S$ | $T$ | $R1$ | $R2$ | $R3$ | $KB$ ( $i$ modelo?)                |
|-----|-----|-----|------|------|------|------------------------------------|
| V   | V   | V   | V    | F    | V    | F                                  |
| V   | V   | F   | V    | F    | V    | F                                  |
| V   | F   | V   | V    | V    | V    | <b>V (modelo <math>M_1</math>)</b> |
| V   | F   | F   | V    | V    | V    | <b>V (modelo <math>M_2</math>)</b> |
| F   | V   | V   | V    | V    | F    | F                                  |
| F   | V   | F   | V    | V    | V    | <b>V (modelo <math>M_3</math>)</b> |
| F   | F   | V   | V    | V    | F    | F                                  |
| F   | F   | F   | F    | V    | V    | F                                  |

Cuadro 1: Tabla de verdad completa.  $KB$  tiene exactamente tres modelos:  $M_1 = (V, F, V)$ ,  $M_2 = (V, F, F)$ ,  $M_3 = (F, V, F)$ .

### a) Tabla de verdad

b)  $KB \models (P \vee S)$ ? En  $M_1$ :  $V \vee F = V$ . En  $M_2$ :  $V \vee F = V$ . En  $M_3$ :  $F \vee V = V$ . Verdadera en los tres únicos modelos de  $KB \Rightarrow KB \models (P \vee S)$ .

c)  $KB \models S$ ? En  $M_1$  y  $M_2$ ,  $S = F$ .  $KB \not\models S$ ; contramodelo:  $M_1 = (P=V, S=F, T=V)$  (o  $M_2$ ), donde  $KB$  es verdadera pero  $S$  es falsa.

d)  $KB \models \neg(S \wedge T)$ ?  $M_1$ :  $S \wedge T = F \wedge V = F \Rightarrow \neg(\cdot) = V$ .  $M_2$ :  $F \wedge F = F \Rightarrow V$ .  $M_3$ :  $V \wedge F = F \Rightarrow V$ . Verdadera en los tres modelos  $\Rightarrow KB \models \neg(S \wedge T)$ .

e)  $KB \models (P \vee T)$ ?  $M_1$ :  $V \vee V = V$ .  $M_2$ :  $V \vee F = V$ .  $M_3$ :  $F \vee F = \mathbf{F}$ . Como  $M_3$  es un modelo de  $KB$  donde  $P \vee T$  es falsa,  $KB \not\models (P \vee T)$ : el administrador **no tiene razón**.  $M_3$  muestra que  $P \vee T$  es **consistente** con  $KB$  (verdadera en  $M_1, M_2$ ) pero no está **implicada** por  $KB$  (falla en  $M_3$ ): consistencia solo exige un modelo testigo, mientras que implicación exige que se cumpla en *todos* los modelos de  $KB$ , sin excepción.

### 2.3. Ejercicio 2: Café de especialidad en el Eje Cafetero

$$\begin{array}{lll}
 R1 : L \wedge H \Rightarrow P & R2 : A \wedge C \Rightarrow PE & R3 : P \wedge PE \Rightarrow CE \\
 R4 : CE \Rightarrow LE & R5 : L \wedge A \Rightarrow SO & \text{Hechos: } L, A, C, H
 \end{array}$$

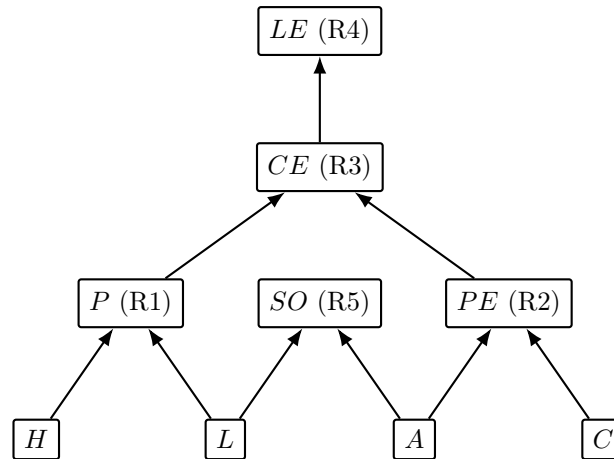


Figura 1: Grafo AND-OR: los hechos  $L, A, C, H$  están en la base; cada símbolo derivado agrupa (AND, implícito por la regla que lo produce) las premisas indicadas por los dos arcos que entran a él. Nótese que  $SO$  no tiene ningún arco de salida: ninguna regla lo usa como premisa.

#### a) Grafo AND-OR

| Paso | Símbolo procesado | Regla que dispara (cuenta a 0)                                           | Agenda al final del paso |
|------|-------------------|--------------------------------------------------------------------------|--------------------------|
| 1    | $L$               | — (decrementa R1: $2 \rightarrow 1$ ; R5: $2 \rightarrow 1$ )            | $[A, C, H]$              |
| 2    | $A$               | R5 dispara $\rightarrow$ agrega $SO$ (decrementa R2: $2 \rightarrow 1$ ) | $[C, H, SO]$             |
| 3    | $C$               | R2 dispara $\rightarrow$ agrega $PE$                                     | $[H, SO, PE]$            |
| 4    | $H$               | R1 dispara $\rightarrow$ agrega $P$                                      | $[SO, PE, P]$            |
| 5    | $SO$              | — ( $SO$ no es premisa de ninguna regla)                                 | $[PE, P]$                |
| 6    | $PE$              | — (decrementa R3: $2 \rightarrow 1$ )                                    | $[P]$                    |
| 7    | $P$               | R3 dispara $\rightarrow$ agrega $CE$                                     | $[CE]$                   |
| 8    | $CE$              | R4 dispara $\rightarrow$ agrega $LE$                                     | $[LE]$                   |
| 9    | $LE$              | <b>es la consulta</b> $\rightarrow$ se retorna verdadero                 | $[]$                     |

Cuadro 2: Traza completa de PL-FC-ENTAILS? para la consulta  $LE$ .

#### b) Traza de forward chaining (agenda FIFO, orden inicial $L, A, C, H$ )

c) **¿Se deriva  $LE$ ?** **Sí.** El hecho derivado  $SO$  (paso 2, vía R5:  $L \wedge A \Rightarrow SO$ ) **no contribuye en nada** a probar  $LE$ :  $SO$  no aparece en la premisa de ninguna regla, así que la cadena que lleva a  $LE$  ( $P, PE \rightarrow CE \rightarrow LE$ ) nunca lo utiliza.

d) **¿Por qué se deriva un hecho irrelevante?** Forward chaining es **dirigido por los datos**: en cada paso dispara *toda* regla cuyas premisas ya se satisfacen, sin usar información sobre cuál es la consulta. Como  $L$  y  $A$  (ya conocidos) satisfacen la premisa de R5 independientemente de que  $SO$  sirva o no para responder  $LE$ , la regla se dispara igual. Esto refleja que el algoritmo es **sano y completo mirando hacia adelante**: deriva absolutamente todo lo que  $KB$  implica, sin ningún mecanismo de poda por relevancia a una meta particular.

### 2.4. Ejercicio 3: La comparsa del Carnaval de Barranquilla

$R1: I \wedge E \Rightarrow H$      $R2: H \wedge V \Rightarrow D$      $R3: G \Rightarrow I$      $R4: B \Rightarrow I$     Hechos:  $B, E, V$

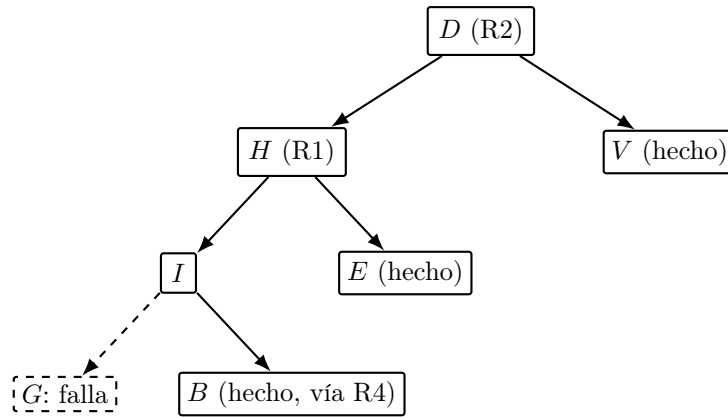


Figura 2: Árbol de prueba de backward chaining para  $D$ . El intento de probar  $I$  vía  $R3$  ( $G \Rightarrow I$ ) falla (rama punteada:  $G$  no es hecho ni cabeza de ninguna regla); se hace backtracking y  $R4$  ( $B \Rightarrow I$ ) rescata la prueba, ya que  $B$  es un hecho.

#### a) Árbol de prueba para la meta $D$

**b) Submeta con backtracking** El backtracking ocurre en la submeta  $I$ : como las reglas se intentan en el orden en que aparecen, primero se prueba **R3** ( $G \Rightarrow I$ ), lo que exige probar  $G$ ; pero  $G$  no es un hecho ni es la conclusión de ninguna otra regla, así que esa rama **falla** sin alternativas. El algoritmo retrocede a la submeta  $I$  y prueba la siguiente regla cuya cabeza es  $I$ : **R4** ( $B \Rightarrow I$ ), que exige probar  $B$ ; como  $B$  es un hecho, **R4 rescata la prueba** y  $I$  queda demostrado.

**c) Comparación con el Ejercicio 2** Forward chaining en el Ejercicio 2 derivó **5 hechos nuevos**:  $SO, PE, P, CE, LE$  (a partir de los 4 hechos iniciales). Backward chaining aquí exploró **7 submetas**:  $D, H, V, I, G, B, E$  (una de ellas,  $G$ , es una rama fallida). Los números son comparables en este ejemplo pequeño, pero la diferencia cualitativa es la que importa:

- **Dirigido por los datos** (forward chaining): parte de los hechos y aplica todas las reglas disparables, sin mirar la meta; puede derivar información verdadera pero irrelevante (como  $SO$ ). Es preferible cuando **no hay una única meta fija** y se esperan muchas consultas sobre la misma base (p. ej. un sistema de monitoreo que deriva conclusiones a medida que llegan datos nuevos).
- **Dirigido por la meta** (backward chaining): parte de la consulta y solo explora lo que es relevante para probarla. Es preferible cuando la base de conocimiento es **grande** pero solo una fracción pequeña es relevante para una consulta puntual (p. ej. sistemas expertos de diagnóstico, o Prolog respondiendo una pregunta específica), aunque puede explorar y descartar ramas fallidas (como  $G$ ) antes de encontrar la regla correcta.

## 2.5. Ejercicio 4: Estación de anillamiento en el páramo

### a) Traducción a LPO

a.1. Todas las aves anilladas en Chingaza migran a Sumapaz:

$$\forall x (Ave(x) \wedge Anillada(x, Chingaza)) \Rightarrow Migra(x, Sumapaz)$$

a.2. Existe un ave anillada en Chingaza que cuida su propio nido:

$$\exists x Ave(x) \wedge Anillada(x, Chingaza) \wedge Cuida(x, Nido(x))$$



a.3. Ningún ave migra a la estación donde fue anillada:

$$\forall x \forall e (Ave(x) \wedge Anillada(x, e)) \Rightarrow \neg Migra(x, e)$$

a.4. Toda ave cuida al menos un nido:

$$\forall x Ave(x) \Rightarrow \exists n Cuida(x, n)$$

## b) Unificación

b.1. *Anillada(Colibri7, x)* y *Anillada(y, Chingaza)*.

Posición 1: *Colibri7* vs. *y*  $\Rightarrow y/Colibri7$ . Posición 2: *x* vs. *Chingaza*  $\Rightarrow x/Chingaza$ .

**UMG:**  $\theta = \{y/Colibri7, x/Chingaza\}$ .

b.2. *Migra(x, Nido(x))* y *Migra(Paramuna3, y)*.

Posición 1: *x* vs. *Paramuna3*  $\Rightarrow x/Paramuna3$  (occurs check: *x* no ocurre en *Paramuna3*, correcto). Aplicando  $\theta_1$ , posición 2: *Nido(Paramuna3)* vs. *y*  $\Rightarrow y/Nido(Paramuna3)$  (occurs check: *y* no ocurre en *Nido(Paramuna3)*).

**UMG:**  $\theta = \{x/Paramuna3, y/Nido(Paramuna3)\}$ .

b.3. *Vecina(Nido(x), x)* y *Vecina(y, Chingaza)*.

Posición 1: *Nido(x)* vs. *y*  $\Rightarrow y/Nido(x)$  (occurs check: *y*  $\neq x$ , no hay ocurrencia). Posición 2: *x* vs. *Chingaza*  $\Rightarrow x/Chingaza$ . Componiendo (se sustituye *x* dentro de la ligadura de *y*): *y/Nido(Chingaza)*.

**UMG:**  $\theta = \{x/Chingaza, y/Nido(Chingaza)\}$ . Verificación: *Vecina(Nido(Chingaza), Chingaza)* en ambos lados.

b.4. *Anillada(Colibri7, Chingaza)* y *Anillada(Colibri7, Sumapaz)*.

Posición 1: *Colibri7* vs. *Colibri7* (coincide). Posición 2: *Chingaza* vs. *Sumapaz*: dos **constantes distintas**.

**No se pueden unificar:** la unificación falla porque no existe sustitución que iguale dos constantes diferentes.

b.5. *Cuida(x, Nido(x))* y *Cuida(Nido(y), y)*.

Posición 1: *x* vs. *Nido(y)*  $\Rightarrow$  occurs check sobre *x*: *x* no aparece en *Nido(y)* (solo aparece *y*), así que *a priori* pasa:  $\theta_1 = \{x/Nido(y)\}$ . Aplicando  $\theta_1$  a la posición 2: *Nido(x)* se convierte en *Nido(Nido(y))*, que debe unificarse con *y*. Posición 2: *Nido(Nido(y))* vs. *y*  $\Rightarrow$  se intentaría *y/Nido(Nido(y))*, pero *y* **ocurre dentro de** *Nido(Nido(y))*.

**No se pueden unificar:** el occurs check falla en la segunda posición (ligar *y* a un término que contiene a *y* generaría el término infinito  $y = Nido(Nido(y)) = Nido(Nido(Nido(y))) = \dots$ ).

**c) Occurs check** El **occurs check** es el paso del algoritmo de unificación que, antes de ligar una variable *v* a un término *t*, verifica que *v* **no aparezca dentro de** *t*. De los pares anteriores, solo **b.5** lo requiere (y falla por su causa): al intentar ligar *y* a *Nido(Nido(y))* se crearía un término circular/infinito, que no es un objeto válido en LPO (los términos deben ser finitos). Un motor de inferencia que omita el occurs check (como muchas implementaciones de Prolog, por razones de eficiencia) aceptaría esta unificación y construiría internamente una estructura de datos **cíclica**; esto puede producir inferencias incorrectas (dos términos que en realidad no denotan el mismo objeto se tratan como unificables) o hacer que el motor entre en un **bucle infinito** al intentar recorrer, imprimir o comparar ese término infinito.

## 2.6. Ejercicio 5: El caso de la pieza extraviada

1.  $\forall x (TieneLlave(x) \wedge TurnoNocturno(x)) \Rightarrow Accede(x, Boveda)$
2.  $\forall x \forall y (Accede(x, Boveda) \wedge Registro(x, y)) \Rightarrow Movio(x, y)$
3.  $TieneLlave(Bruno)$       4.  $TurnoNocturno(Bruno)$       5.  $Registro(Bruno, Poporo)$
6.  $TieneLlave(Amparo)$       7.  $TurnoNocturno(Celia)$        $\alpha = \exists x Movio(x, Poporo)$

**a) Forma clausal (variables estandarizadas)** Se aplican los pasos de conversión a FNC de la definición anterior a cada sentencia.

**Sentencia 1** ( $TieneLlave(x) \wedge TurnoNocturno(x) \Rightarrow Accede(x, Boveda)$ ):

**Paso 1** (eliminar  $\Rightarrow$ ):

$$\forall x \neg(TieneLlave(x) \wedge TurnoNocturno(x)) \vee Accede(x, Boveda)$$

**Paso 2** (De Morgan):

$$\forall x (\neg TieneLlave(x) \vee \neg TurnoNocturno(x)) \vee Accede(x, Boveda)$$

**Paso 3** (estandarizar):  $x \rightarrow x_1$ .

**Pasos 4–6** (no hay  $\exists$  que skolemizar; se elimina  $\forall$ ; ya es disyunción):

$$C_1 : \neg TieneLlave(x_1) \vee \neg TurnoNocturno(x_1) \vee Accede(x_1, Boveda)$$

**Sentencia 2** ( $Accede(x, Boveda) \wedge Registro(x, y) \Rightarrow Movio(x, y)$ ): por el mismo procedimiento (eliminar  $\Rightarrow$ , De Morgan, estandarizar  $x, y \rightarrow x_2, y_2$ , sin existenciales que skolemizar), se obtiene directamente

$$C_2 : \neg Accede(x_2, Boveda) \vee \neg Registro(x_2, y_2) \vee Movio(x_2, y_2).$$

**Hechos** (ya son cláusulas unitarias, sin cuantificadores ni conectivos que eliminar):

$$C_3 : TieneLlave(Bruno) \quad C_4 : TurnoNocturno(Bruno) \quad C_5 : Registro(Bruno, Poporo)$$

$$C_6 : TieneLlave(Amparo) \quad C_7 : TurnoNocturno(Celia)$$

**Negación de la meta**  $\alpha = \exists x Movio(x, Poporo)$ :

**Paso 0** (negar):  $\neg\alpha = \neg\exists x Movio(x, Poporo)$ .

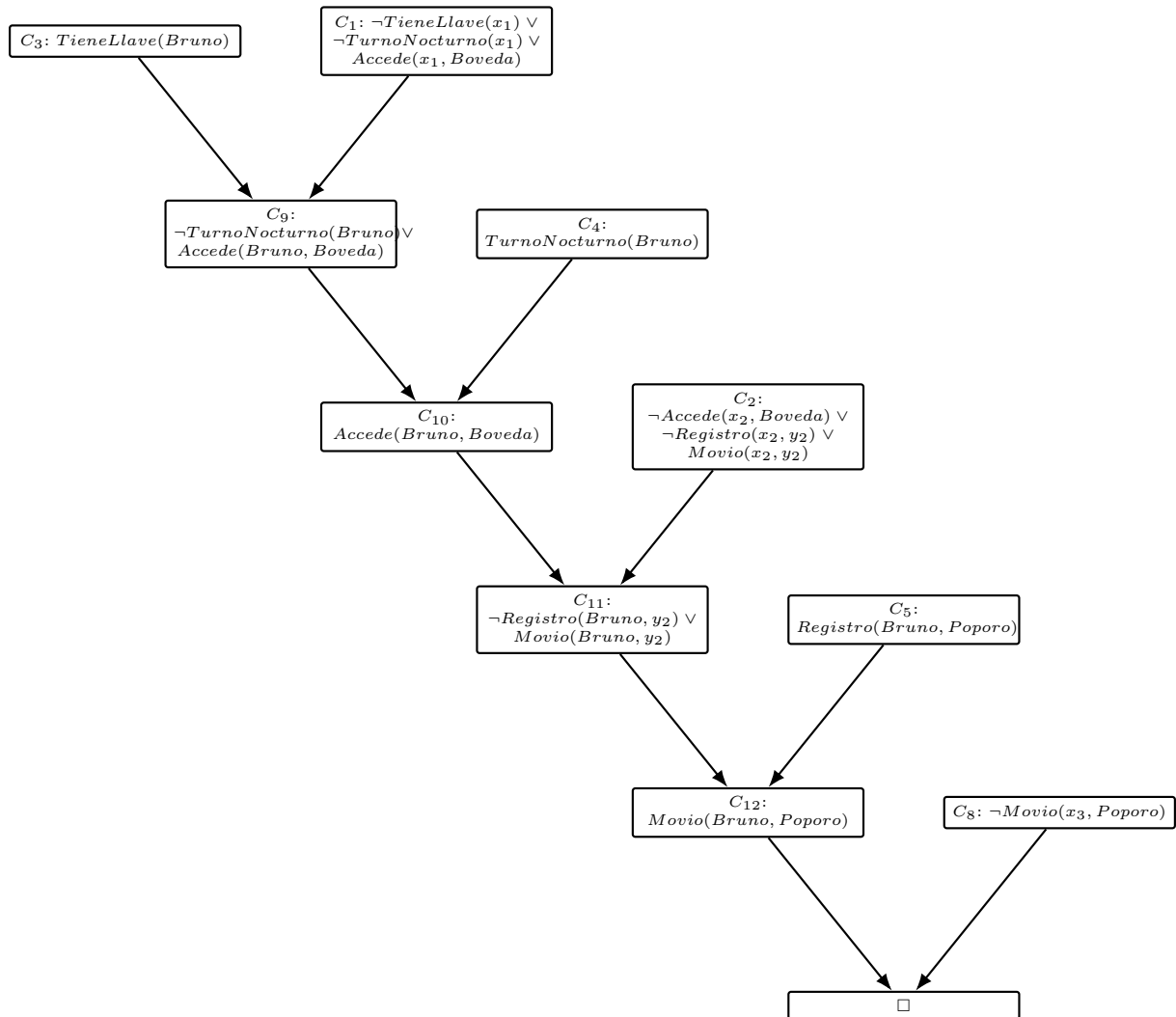
**Paso 2** (mover  $\neg$  hacia adentro,  $\neg\exists x \Phi \equiv \forall x \neg\Phi$ ):  $\forall x \neg Movio(x, Poporo)$ .

**Paso 3** (estandarizar):  $x \rightarrow x_3$ .

**Pasos 4–6** (no hay  $\exists$  que skolemizar; se elimina  $\forall$ ):  $C_8 : \neg Movio(x_3, Poporo)$ .

Nótese que  $\exists x$  en la meta *original*  $\alpha$  no se skolemiza directamente: se skolemiza la variable existencial *después de negar*, y aquí la negación la convirtió en universal, así que no aparece ninguna constante de Skolem en  $C_8$  (a diferencia del Ejercicio 4.a.2, donde el  $\exists x$  sí quedaría como variable libre estandarizada si se llevara a FNC, sin universales que lo envuelvan).

| Paso | Cláusulas resueltas | MGU                              | Resolvente                                                   |
|------|---------------------|----------------------------------|--------------------------------------------------------------|
| 1    | $C_3$ con $C_1$     | $\{x_1/Bruno\}$                  | $C_9 : \neg TurnoNocturno(Bruno) \vee Accede(Bruno, Boveda)$ |
| 2    | $C_4$ con $C_9$     | $\{\}$ (ya coincide en $Bruno$ ) | $C_{10} : Accede(Bruno, Boveda)$                             |
| 3    | $C_{10}$ con $C_2$  | $\{x_2/Bruno\}$                  | $C_{11} : \neg Registro(Bruno, y_2) \vee Movio(Bruno, y_2)$  |
| 4    | $C_5$ con $C_{11}$  | $\{y_2/Poporo\}$                 | $C_{12} : Movio(Bruno, Poporo)$                              |
| 5    | $C_{12}$ con $C_8$  | $\{x_3/Bruno\}$                  | $\square$ (cláusula vacía)                                   |

Cuadro 3: Refutación: se deriva  $\square$  a partir de  $KB \wedge \neg\alpha$ .Figura 3: Árbol de refutación completo: cada resolvente combina los dos nodos que apuntan hacia él, hasta llegar a  $\square$ .

## b) Refutación por resolución

c) **Extracción de la respuesta** La variable existencial de la meta original ( $x$  en  $\exists x \text{ Movio}(x, \text{Poporo})$ ) se estandarizó como  $x_3$  en  $C_8$ . Acumulando las sustituciones a lo largo de la refutación,  $x_3$  termina ligada a  $Bruno$  (paso 5). **Respuesta: Bruno movió el poporo.**

#### d) Por qué Amparo y Celia no cierran una refutación alternativa

- **Amparo** ( $C_6: TieneLlave(Amparo)$ ): resolviendo con  $C_1 (x_1/Amparo)$  se obtiene

$$\neg TurnoNocturno(Amparo) \vee Accede(Amparo, Boveda).$$

Para continuar haría falta resolver el literal  $\neg TurnoNocturno(Amparo)$  contra una cláusula unitaria  $TurnoNocturno(Amparo)$ , pero esa cláusula **no existe** en la KB (el enunciado aclara que Amparo no estuvo en el turno nocturno): la cadena se detiene ahí.

- **Celia** ( $C_7: TurnoNocturno(Celia)$ ): resolviendo con  $C_1 (x_1/Celia)$  se obtiene

$$\neg TieneLlave(Celia) \vee Accede(Celia, Boveda).$$

Aquí falta una cláusula unitaria  $TieneLlave(Celia)$  para resolver  $\neg TieneLlave(Celia)$ ; tampoco existe (el enunciado aclara que Celia no tiene llave). Igual que con Amparo, no hay literal complementario que unifique.

- Incluso si, hipotéticamente, se lograra derivar  $Accede(Amparo, Boveda)$  o  $Accede(Celia, Boveda)$ , el segundo paso exigiría resolver contra  $C_2$  y luego necesitaría una cláusula  $Registro(Amparo, y)$  o  $Registro(Celia, y)$  para producir *Movio*; la única cláusula *Registro* de la KB es  $Registro(Bruno, Poporo)$  así que tampoco hay unificador posible ahí.

e) **¿Por qué probar  $KB \wedge \neg\alpha$  insatisfacible en vez de derivar  $\alpha$  directamente?** Probar  $KB \models \alpha$  “directamente” exigiría verificar que  $\alpha$  es verdadera en *todo* modelo de  $KB$  —en LPO, con dominios potencialmente infinitos, no hay forma de enumerar todos los modelos. La refutación invierte el problema: por el **teorema de refutación** ( $KB \models \alpha \Leftrightarrow KB \wedge \neg\alpha$  insatisfacible), basta con encontrar *una* contradicción sintáctica finita (la cláusula vacía  $\square$ ) para concluir que **ningún** modelo puede satisfacer  $KB$  y  $\neg\alpha$  a la vez, es decir, que todo modelo de  $KB$  satisface  $\alpha$ . El principio que conecta ambas cosas es precisamente ese teorema de refutación, sustentado por el **teorema de completitud** de la resolución: convierte una pregunta semántica sobre *todos los modelos* en una búsqueda sintáctica, finita y mecánicamente verificable, de una contradicción.

### 3. Planeación: Definición y Algoritmos Generales

#### 3.1. ¿Qué es la Planeación Clásica?

**Definición 3.1** (Planeación clásica). La **planeación clásica** es la tarea de hallar una secuencia de acciones que lleve a un agente desde un estado inicial hasta un estado que satisface una meta, en ambientes **discretos, determinísticos, estáticos y completamente observables**.

**Aproximaciones previas y sus limitaciones:**

- **Agente de solución de problemas** (búsqueda genérica): requiere una **función heurística** diseñada a mano para cada nuevo dominio.
- **Agente de lógica proposicional híbrido**: requiere codificar el problema completo como sentencias proposicionales, con una **representación explícita** de un espacio de estados que crece exponencialmente.

**Ejemplo 3.1** (Por qué la representación importa: comprar libros en línea). Si se define una acción de compra distinta por cada código ISBN de 10 dígitos, un agente sin estructura de dominio debe considerar del orden de 10 mil millones de acciones. Para comprar *cuatro* libros distintos (meta  $Tener(A) \wedge Tener(B) \wedge Tener(C) \wedge Tener(D)$ ), el número de planes de cuatro pasos a examinar sin explotar la estructura del problema es del orden de  $10^{40}$ . Sin embargo, un planificador que sepa que  $Tener(c) \Rightarrow Comprar(c)$  es una regla **independiente por cada código**  $c$  resuelve el problema de forma trivial. Esto motiva una representación **factorizada e independiente de dominio**: PDDL.

### 3.2. PDDL: Representación de Estados

PDDL (*Planning Domain Definition Language*) representa estados, acciones, metas y dominios de forma factorizada: los mismos algoritmos de planeación funcionan para cualquier problema descrito en PDDL, sin heurísticas ad hoc por dominio.

**Definición 3.2** (Fluente atómico definido). Un **estado** se representa como una conjunción de **fluentes atómicos definidos**:

- **Fluente**: un aspecto del mundo que puede cambiar con el tiempo.
- **Atómico**: un único predicado aplicado a argumentos (no hay  $\wedge$ ,  $\vee$  ni  $\Rightarrow$  dentro del fluente mismo).
- **Definido**: los argumentos son **constantes** (sin variables) y el fluente no está negado.

Ejemplos válidos:  $Pobre \wedge Desconocido$ ;  $En(Camion_1, Melbourne) \wedge En(Camion_2, Sydney)$ .

No permitidos:  $En(x, y)$  (tiene variables),  $\neg Pobre$  (tiene negación).

PDDL usa una **base de datos semántica**: todo fluente que no aparece explícitamente en la descripción de un estado se asume **falso** (suposición de mundo cerrado), y constantes distintas (p. ej.  $Camion_1, Camion_2$ ) se asumen objetos **distintos entre sí** (suposición de nombres únicos).

### 3.3. PDDL: Schema de Acción

**Definición 3.3** (Schema de acción). Un **schema de acción** representa toda una **familia** de acciones mediante variables. Sus elementos son: un **nombre**, una **lista de variables**, una **precondición** y un **efecto**, ambos conjunciones de literales (positivos o negativos).

**Ejemplo (schema, con variables):**

Acción( $Volar(p, desde, hacia)$ )

PRECOND :  $En(p, desde) \wedge Avión(p) \wedge Aeropuerto(desde) \wedge Aeropuerto(hacia)$

EFECTO :  $\neg En(p, desde) \wedge En(p, hacia)$

**La misma acción, instanciada con constantes:**

Acción( $Volar(P_1, SFO, JFK)$ )

PRECOND :  $En(P_1, SFO) \wedge Avión(P_1) \wedge Aeropuerto(SFO) \wedge Aeropuerto(JFK)$

EFECTO :  $\neg En(P_1, SFO) \wedge En(P_1, JFK)$

Listing 1: El mismo schema de acción en PDDL

```

1 (:action Fly
2   :parameters (?p - Plane ?from ?to - Airport)
3   :precondition (and (At ?p ?from) (Plane ?p)
4                     (Airport ?from) (Airport ?to))
5   :effect (and (At ?p ?to) (not (At ?p ?from))))

```

**Definición 3.4** (Acción aplicable y RESULTADO). Una acción  $a$  es **aplicable** en un estado  $s$  si cada literal positivo de su precondición está en  $s$  y ningún literal negado de su precondición está en  $s$  (equivalentemente: la precondición se determina unificando sus literales con los fluentes de  $s$ , sustituyendo variables por constantes). El estado resultante de ejecutar  $a$  en  $s$  es

$$\text{RESULTADO}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a),$$

donde  $\text{DEL}(a)$  son los fluentes que aparecen como literales **negados** en el efecto de  $a$  (se eliminan de  $s$ ) y  $\text{ADD}(a)$  son los que aparecen como literales **positivos** (se añaden a  $s$ ).

Un **dominio de planeación** es un conjunto de schemas de acción. Un **problema** específico en ese dominio agrega un **estado inicial** (INICIAR, conjunción de fluentes definidos) y una **meta** (OBJETIVO, conjunción de literales positivos o negativos que *sí* puede contener variables, interpretadas existencialmente): por ejemplo,  $\text{OBJETIVO}(En(C_1, SFO) \wedge \neg En(C_2, SFO) \wedge En(p, SFO))$  se satisface con cualquier avión  $p$  que esté en SFO, siempre que  $C_1$  esté en SFO y  $C_2$  no lo esté.

### 3.4. Ejemplos de Dominios Clásicos

**Transporte de carga aérea.** Predicados:  $En(c, p)$  (el cargo  $c$  está dentro del avión  $p$ ),  $En(x, a)$  (el objeto  $x$  está en el aeropuerto  $a$ ),  $Dentro(c, p)$ . Como PDDL no cuenta con un cuantificador universal para expresar “todo el cargo dentro del avión viaja con él”, la estrategia es que un cargo **deja de estar** en su aeropuerto ( $En$ ) mientras está dentro de un avión, y solo vuelve a tener una ubicación ( $En$ ) en el aeropuerto de destino.

Acción( $Cargar(c, p, a)$ )

PRECOND :  $En(c, a) \wedge En(p, a) \wedge Cargo(c) \wedge Avión(p) \wedge Aeropuerto(a)$

EFECTO :  $\neg En(c, a) \wedge Dentro(c, p)$

Acción( $Descargar(c, p, a)$ )

PRECOND :  $Dentro(c, p) \wedge En(p, a) \wedge Cargo(c) \wedge Avión(p) \wedge Aeropuerto(a)$

EFECTO :  $En(c, a) \wedge \neg Dentro(c, p)$

Con el siguiente estado inicial:

INICIAR( $En(C_1, SFO) \wedge En(C_2, JFK) \wedge En(P_1, SFO) \wedge En(P_2, JFK)$

$\wedge Cargo(C_1) \wedge Cargo(C_2) \wedge Avión(P_1) \wedge Avión(P_2) \wedge Aeropuerto(JFK) \wedge Aeropuerto(SFO)$ )

y la siguiente meta:

OBJETIVO( $En(C_1, JFK) \wedge En(C_2, SFO)$ )

una solución es

$[Cargar(C_1, P_1, SFO), Volar(P_1, SFO, JFK), Descargar(C_1, P_1, JFK),$   
 $Cargar(C_2, P_2, JFK), Volar(P_2, JFK, SFO), Descargar(C_2, P_2, SFO)].$

**Neumático de repuesto.** Acciones  $Quitar(x, y)$  (quitar el objeto  $x$  de  $y$ ),  $Colocar(x, y)$ , y  $DejarDeNoche$ : sin precondiciones, con el efecto de **borrar** todo lo que se sabía sobre la ubicación de las ruedas, modelando que en un barrio peligroso las llantas pueden desaparecer si el carro queda sin vigilancia.

Acción( $Quitar(Repuesto, Maletero)$ ), PRECOND :  $En(Repuesto, Maletero)$

EFECTO :  $\neg En(Repuesto, Maletero) \wedge En(Repuesto, Suelo)$

Acción( $Colocar(Repuesto, Eje)$ ), PRECOND :  $En(Repuesto, Suelo) \wedge \neg En(Deshinchada, Eje)$

EFECTO :  $\neg En(Repuesto, Suelo) \wedge En(Repuesto, Eje)$

Con el siguiente estado inicial y meta:

INICIAR( $En(Deshinchada, Eje) \wedge En(Repuesto, Maletero)$ )

OBJETIVO( $En(Repuesto, Eje)$ )

una solución es

$[Quitar(Repuesto, Maletero), Quitar(Deshinchada, Eje), Colocar(Repuesto, Eje)].$

**Mundo de bloques.** Predicados:  $Sobre(b, x)$  (el bloque  $b$  está sobre  $x$ , que puede ser otro bloque o la mesa),  $Despejar(b)$  (no hay ningún bloque sobre  $b$ ),  $Bloque(b)$ . Una sola acción  $Mover(b, x, y)$  no basta: al mover el *último* bloque de una pila hacia la mesa no hay ningún bloque  $y$  concreto que reciba el efecto  $Despejar(y)$ , así que PDDL usa una segunda acción dedicada,  $MoverSobreTablero(b, x)$ , para mover un bloque directamente a la mesa.

Acción( $Mover(b, x, y)$ )

PRECOND :  $Sobre(b, x) \wedge Despejar(b) \wedge Despejar(y) \wedge Bloque(b) \wedge (b \neq x) \wedge (b \neq y) \wedge (x \neq y)$

EFEECTO :  $Sobre(b, y) \wedge Despejar(x) \wedge \neg Sobre(b, x) \wedge \neg Despejar(y)$

Acción( $MoverSobreTablero(b, x)$ )

PRECOND :  $Sobre(b, x) \wedge Despejar(b) \wedge Bloque(b) \wedge (b \neq x)$

EFEECTO :  $Sobre(b, Mesa) \wedge Despejar(x) \wedge \neg Sobre(b, x)$

Con el siguiente estado inicial:

INICIAR( $Sobre(A, Mesa) \wedge Sobre(B, Mesa) \wedge Sobre(C, Mesa)$

$\wedge Bloque(A) \wedge Bloque(B) \wedge Bloque(C) \wedge Despejar(A) \wedge Despejar(B) \wedge Despejar(C)$ )

y la siguiente meta:

OBJETIVO( $Sobre(A, B) \wedge Sobre(B, C)$ )

una solución de dos pasos es  $[Mover(B, Mesa, C), Mover(A, Mesa, B)]$ .

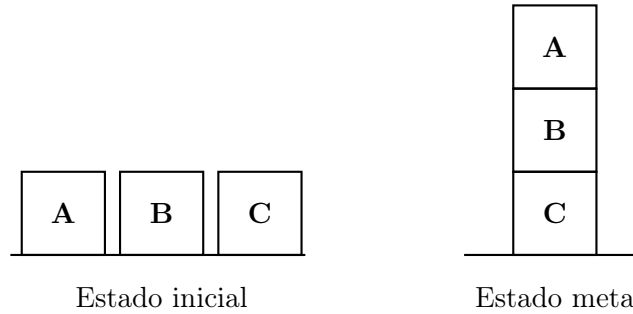


Figura 4: Mundo de bloques: en el estado inicial  $A$ ,  $B$  y  $C$  están cada uno directamente sobre la mesa; la meta apila  $A$  sobre  $B$  sobre  $C$ . La solución  $[Mover(B, Mesa, C), Mover(A, Mesa, B)]$  primero coloca  $B$  sobre  $C$  y luego  $A$  sobre  $B$ .

### 3.5. Búsqueda hacia Adelante en el Espacio de Estados (Progresión)

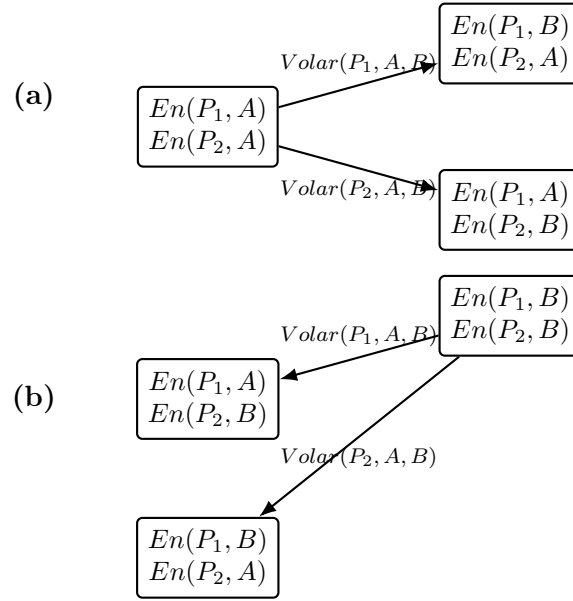


Figura 5: (a) Búsqueda hacia adelante: desde el estado inicial, cada avión disponible genera una rama distinta al aplicar *Volar*. (b) Búsqueda hacia atrás: desde una meta con ambos aviones en *B*, cada regresión sobre *Volar* genera un estado predecesor distinto, según cuál avión se supone que ya voló.

La **planeación de progresión** aplica los algoritmos de búsqueda genéricos (Cap. 3–4) directamente sobre PDDL: parte del estado inicial (conjunto de literales positivos, con suposición de mundo cerrado) y en cada estado  $s$  genera  $ACCIONES(s)$  unificando las precondiciones de cada schema con los flujos de  $s$ , sustituyendo variables por las constantes que hacen coincidir la unificación (instanciación definida).

- **Problema:** un schema con varios literales en su precondición puede unificar de *muchas* formas distintas. En el dominio del neumático, la precondición  $En(obj, loc)$  de *Quitar* unifica tanto con  $\{obj/Deshinchada, loc/Eje\}$  como con  $\{obj/Repuesto, loc/Maletero\}$ , generando ramas adicionales por cada instanciación posible.
- **Ejemplo de escala (carga aérea):** con 10 aeropuertos, 5 aviones y 20 piezas de cargo por aeropuerto (50 aviones y 200 cargas en total, cada avión puede volar a 9 aeropuertos y cada carga puede cargarse/descargarse), hay entre 450 y 10.450 acciones aplicables por estado ( $\approx 2000$  en promedio). La meta de mover toda la carga de un aeropuerto a otro requiere una solución de **41 pasos**, lo que da un árbol de búsqueda de hasta 2000<sup>41</sup> nodos: la búsqueda ciega es completamente inviable, y motiva las **heurísticas independientes de dominio** de la Clase 3.

### 3.6. Búsqueda hacia Atrás en el Espacio de Estados (Regresión)

La **planeación de regresión** parte de la meta y aplica acciones “hacia atrás” hasta llegar a un estado que coincide con el inicial, considerando en cada paso únicamente las **acciones relevantes**.

**Definición 3.5** (Acción relevante). Una acción  $a$  es **relevante** para una meta  $g$  si el efecto de  $a$  unifica con *al menos un* literal de  $g$ , **sin negar ninguna otra parte** de  $g$ .



**Ejemplo:** con meta  $\neg Pobre \wedge Famoso$ , una acción con efecto *Famoso* es relevante. Una acción con efecto  $Pobre \wedge Famoso$  **no** es relevante –aunque también produzca *Famoso*– porque su efecto haría verdadero *Pobre*, contradiciendo  $\neg Pobre$  en la meta.

Dada una meta  $g$  y una acción relevante  $a$ , la **regresión** de  $g$  sobre  $a$  produce una nueva meta (estado predecesor)  $g'$ :

$$\text{POS}(g') = (\text{POS}(g) - \text{ADD}(a)) \cup \text{POS}(\text{PRECOND}(a))$$

$$\text{NEG}(g') = (\text{NEG}(g) - \text{DEL}(a)) \cup \text{NEG}(\text{PRECOND}(a))$$

Intuición: los literales que  $a$  ya garantiza se **remueven** de la meta (ya quedarán satisfechos al ejecutar  $a$ ), y se **añaden** las precondiciones de  $a$ , porque deben cumplirse *antes* de ejecutarla –si no,  $a$  no podría haberse ejecutado.

**Ejemplo:** meta  $En(C_2, SFO)$ , schema *Descargar* con efecto  $En(c, a)$ . Unificando el objetivo con el efecto se obtiene  $\theta = \{c/C_2, a/SFO\}$ ; tras estandarizar  $p \rightarrow p'$ :

$$\text{Acción}(\text{Descargar}(C_2, p', SFO))$$

$$\text{PRECOND} : \text{Dentro}(C_2, p') \wedge En(p', SFO) \wedge \text{Cargo}(C_2) \wedge \text{Avión}(p') \wedge \text{Aeropuerto}(SFO)$$

$$\text{EFECTO} : En(C_2, SFO) \wedge \neg \text{Dentro}(C_2, p')$$

La nueva meta (estado predecesor) es

$$g' = \text{Dentro}(C_2, p') \wedge En(p', SFO) \wedge \text{Cargo}(C_2) \wedge \text{Avión}(p') \wedge \text{Aeropuerto}(SFO).$$

**Comparación con la búsqueda hacia adelante:** suponga la meta  $Tener(9780134610993)$  con el schema  $\text{Acción}(\text{Comprar}(i))$ ,  $\text{PRECOND} : ISBN(i)$ ,  $\text{EFECTO} : Tener(i)$ . Sin heurística, la búsqueda hacia **adelante** tendría que enumerar del orden de 10 mil millones de acciones *Comprar* definidas para encontrar la correcta. La búsqueda hacia **atrás** solo unifica la meta con el efecto ( $\theta = \{i'/9780134610993\}$ ) y obtiene el estado predecesor directamente en un paso:  $ISBN(9780134610993)$ . Esto ilustra por qué la regresión suele ser más eficiente cuando la meta es específica y el espacio de acciones aplicables hacia adelante es enorme.

### 3.7. Planeación como Satisfacibilidad Booleana (SATPLAN)

**Estrategia:** fijar un horizonte de tiempo  $T$  y codificar *todo* el problema de planeación (con ese horizonte) como una única fórmula proposicional grande, que se entrega a un solucionador SAT; un modelo satisfactible codifica un plan válido.

1. **Proposicionalizar las acciones:** por cada schema, sustituir sus variables por constantes y añadir un superíndice de tiempo  $t$ : p. ej.  $\text{Volar}P_1SFOJFK^1$ .
2. **Axiomas de exclusión de acciones:** para cada par de acciones que no pueden ocurrir simultáneamente,

$$\neg(\text{Volar}P_1SFOJFK^1 \wedge \text{Volar}P_1SFOBUEH^1)$$

(el mismo avión no puede volar a dos destinos distintos en el mismo paso de tiempo).

3. **Axiomas de precondición:** por cada acción definida  $A^t$ ,

$$A^t \Rightarrow \text{PRE}(A)^t$$

(si la acción ocurrió en el tiempo  $t$ , sus precondiciones eran verdaderas en  $t$ ). Por ejemplo,  $\text{Volar}P_1SFOJFK^1 \Rightarrow En(P_1, SFO)^1 \wedge \text{Avión}(P_1)^1 \wedge \text{Aeropuerto}(SFO)^1 \wedge \text{Aeropuerto}(JFK)^1$ .

4. **Axiomas de estado sucesor:** por cada fluente  $F$ ,

$$F^{t+1} \Leftrightarrow \text{ACCIONCAUSAF}^t \vee (F^t \wedge \neg \text{ACCIONCAUSANOF}^t),$$

donde  $\text{ACCIONCAUSAF}$  es la disyunción de todas las acciones definidas que **añaden**  $F$ , y  $\text{ACCIONCAUSANOF}$  la disyunción de las que lo **eliminan**.

5. **Estado inicial:** se asevera  $F^0$  por cada fluente verdadero en el problema inicial, y  $\neg F^0$  por cada fluente no mencionado.
6. **Meta proposicionalizada:** disyunción sobre todas las instancias definidas donde las variables de la meta se reemplazan por constantes. Por ejemplo, la meta  $\text{En}(A, x) \wedge \text{Bloque}(x)$  con objetos  $A, B, C$  se convierte en

$$(\text{En}(A, A) \wedge \text{Bloque}(A)) \vee (\text{En}(A, B) \wedge \text{Bloque}(B)) \vee (\text{En}(A, C) \wedge \text{Bloque}(C)).$$

### 3.8. Otras Estrategias de Planeación

- **Grafo de planeación:** estructura de capas alternadas de literales y acciones ( $S_0, A_0, S_1, A_1, \dots$ ) que se construye en tiempo polinomial y acota qué literales son alcanzables en cuántos pasos. Se retoma en la **Clase 3** como base de la heurística FF y de las heurísticas de planeación en general.

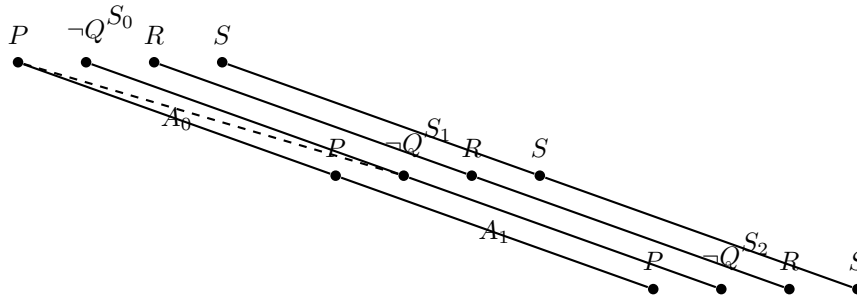


Figura 6: Grafo de planeación (esquemático): las líneas sólidas verticales son *acciones de persistencia* (un literal verdadero puede seguir siéndolo sin ninguna acción que lo cambie); la línea punteada representa una acción real que conecta una precondition en  $S_0$  con un efecto en  $S_1$ . Cada capa  $S_i$  acumula todos los literales alcanzables en  $i$  pasos o menos.

- **Cálculo situacional** (*situation calculus*): representa la planeación directamente en LPO mediante un término  $do(a, s)$  que denota el estado resultante de ejecutar la acción  $a$  en la situación  $s$  (p. ej.  $do(\text{Move}(\text{rob}, o109, o103), \text{init})$ ). Permite razonar con todo el aparato de LPO (cuantificadores, igualdad, teoría de conjuntos), a costa de perder la estructura factorizada –y por tanto buena parte de la eficiencia– que ofrece PDDL.
- **Planeación de orden parcial (POP)**: en vez de comprometerse con un **orden total y rígido** entre las acciones del plan (*fixed order*), POP solo impone las restricciones de orden estrictamente necesarias (*least commitment*), dejando sin ordenar entre sí las acciones que no dependen una de otra.

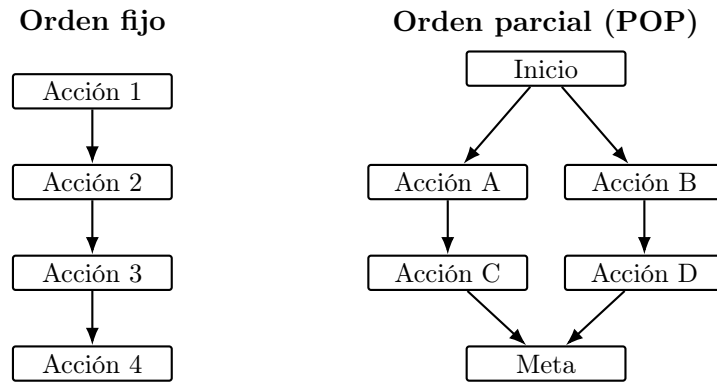


Figura 7: Orden fijo vs. orden parcial: en el plan de orden fijo cada acción depende rígidamente de la anterior; en POP, las acciones A/B (y C/D) son mutuamente independientes y solo se ordenan respecto a lo que sí exigen sus dependencias reales, dejando margen para ejecutarlas en cualquier orden entre sí o en paralelo.

## 4. Planeación: Heurísticas y Planeación Jerárquica

La Clase 2 mostró que la búsqueda ciega sobre PDDL no escala (hasta 2000<sup>41</sup> nodos en el ejemplo de carga aérea). Esta clase repasa qué hace admisible a una heurística y luego construye heurísticas **independientes de dominio** para planeación por relajación del problema, un conjunto de técnicas de **podado independiente de dominio**, y finalmente la **planeación jerárquica (HTN)**, que ataca el problema de escala organizando acciones en niveles de abstracción en vez de heurísticas.

### 4.1. Repaso: Admisibilidad y Monotonicidad de Heurísticas

**Definición 4.1** (Heurística). Una **heurística**  $h(n)$  es una función que estima qué tan cerca está un estado (nodo  $n$ ) de la meta, y sirve para guiar la búsqueda hacia el camino con mayor promesa. Ejemplos clásicos: distancia Manhattan, distancia euclidiana.

Evaluar una heurística no es solo verificar que *encuentre* una solución, sino que encuentre el **camino más corto**. Tres preguntas clave:

- **Admisibilidad:** ¿la heurística nunca sobrestima el costo real? Formalmente,

$$0 \leq h(n) \leq h^*(n),$$

donde  $h^*(n)$  es el costo real óptimo desde  $n$  hasta la meta. Una heurística admisible garantiza encontrar el camino más corto (cuando existe solución).

- **Informedness:** dadas dos heurísticas admisibles, ¿cuál domina a la otra (da estimados más ajustados, y por tanto expande menos nodos)?
- **Monotonicidad / consistencia:** si ya se expandió un estado, ¿se garantiza que no se va a encontrar el mismo estado después por un camino más corto? Una heurística es **consistente** si, para todo nodo  $n_i$  y sucesor  $n_j$  alcanzado con una acción de costo  $\text{COSTO}(n_i, n_j)$ ,

$$h(n_i) - h(n_j) \leq \text{COSTO}(n_i, n_j).$$

## 4.2. Heurísticas en Planeación: Relajación de Problemas

**Diferencia clave frente a búsqueda genérica:** en planeación, las heurísticas pueden construirse de forma **independiente del dominio**, porque PDDL ya factoriza el problema en flujos y schemas de acción –no hace falta que un experto diseñe la heurística a mano para cada dominio nuevo.

Un problema de búsqueda se modela como un grafo: nodos = estados, arcos = acciones, objetivo = estado meta. La estrategia general es la **relajación**: construir una versión más fácil del problema cuyo costo exacto (o una aproximación de él) sirve como heurística admisible para el problema original. Dos formas de relajar:

- **Agregar más arcos al grafo** (permitir más transiciones entre estados) –heurística de ignorar precondiciones.
- **Agrupar múltiples nodos en uno solo** (menos estados) –abstracción de estado.

## 4.3. Heurística de Ignorar Precondiciones

**Definición 4.2** (Ignorar precondiciones). Se remueven todas las precondiciones de las acciones: toda acción queda aplicable en cualquier estado. En el problema relajado, cada fuente de la meta se alcanza en un solo paso (o el problema es imposible). El número de pasos para resolver el problema relajado es, aproximadamente, el número de metas todavía insatisfechas.

**Estrategia general:** contar el mínimo número de acciones tal que la unión de sus efectos satisfaga la meta. Esto es exactamente una **instancia del problema de cobertura de conjuntos** (*set cover*): dado un universo  $\{1, \dots, m\}$  y  $n$  conjuntos cuya unión cubre el universo, hallar el menor número de conjuntos cuya unión también lo cubre.

**Ejemplo 4.1** (Cobertura de conjuntos). Sea  $U = \{1, 2, 3, 4, 5\}$  y  $S = \{\{1, 2, 3\}, \{4, 5\}\}$ : basta escoger los dos conjuntos de  $S$  para cubrir  $U$ .

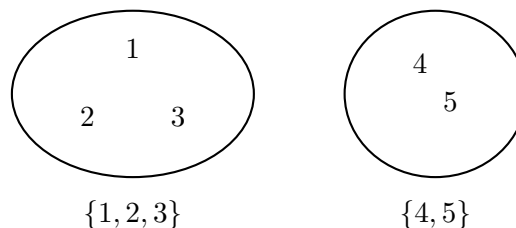


Figura 8:  $U = \{1, 2, 3, 4, 5\}$  cubierto por  $S = \{\{1, 2, 3\}, \{4, 5\}\}$ : la cobertura mínima requiere ambos conjuntos.

El problema de cobertura mínima es **NP-Hard** en general, pero un algoritmo voraz simple (cobertura por conjuntos) devuelve una cobertura dentro de un factor  $\log n$  de la mínima real, donde  $n$  es el número de literales en la meta –suficiente para una heurística útil, aunque no exacta.

**Paso a paso del algoritmo voraz sobre el ejemplo anterior:**

1. Elementos sin cubrir:  $\{1, 2, 3, 4, 5\}$ .
2. Escoger el conjunto de  $S$  que cubre **más** elementos sin cubrir todavía:  $\{1, 2, 3\}$  cubre 3 elementos,  $\{4, 5\}$  cubre 2; se escoge  $\{1, 2, 3\}$ .
3. Elementos sin cubrir restantes:  $\{4, 5\}$ .

4. Escoger de nuevo el conjunto que más cubre: solo queda  $\{4, 5\}$ , que cubre los 2 restantes; se escoge.
5. Elementos sin cubrir:  $\emptyset \Rightarrow$  se detiene. Cobertura hallada:  $\{\{1, 2, 3\}, \{4, 5\}\}$ , de tamaño 2 –que en este ejemplo coincide con la cobertura mínima real.

**Ejemplo 4.2** (8-puzzle). Con el schema

$$\text{Acción}(\text{Mover}(t, s_1, s_2))$$

$$\text{PRECOND} : \text{En}(t, s_1) \wedge \text{Baldosa}(t) \wedge \text{Vacía}(s_2) \wedge \text{Adyacente}(s_1, s_2)$$

$$\text{EFECTO} : \text{En}(t, s_2) \wedge \text{Vacía}(s_1) \wedge \neg \text{En}(t, s_1) \wedge \neg \text{Vacía}(s_2)$$

dos relajaciones distintas producen dos heurísticas clásicas:

- Remover  $\text{Vacía}(s_2) \wedge \text{Adyacente}(s_1, s_2)$ : cualquier baldosa se puede mover a cualquier casilla en una sola acción  $\Rightarrow$  heurística del **número de baldosas fuera de lugar**.
- Remover solo  $\text{Vacía}(s_2)$  (se conserva  $\text{Adyacente}$ ): cada baldosa solo se puede mover a casillas adyacentes, pero sin bloqueos  $\Rightarrow$  heurística de **distancia Manhattan**.

**Ejemplo numérico (informedness)**: supóngase un estado con exactamente 3 baldosas fuera de su posición meta.

1. Heurística  $h_1$  (baldosas fuera de lugar): cuenta directamente las baldosas mal ubicadas, sin importar qué tan lejos estén  $\Rightarrow h_1 = 3$ .
2. Heurística  $h_2$  (distancia Manhattan): suma, para cada baldosa fuera de lugar, el número de casillas que la separan (en horizontal + vertical) de su posición meta. Si esas 3 baldosas están, respectivamente, a 1, 2 y 2 pasos de su posición meta,  $h_2 = 1 + 2 + 2 = 5$ .
3. Como  $h_2 \geq h_1$  siempre (mover una baldosa fuera de lugar toma al menos 1 paso, y puede tomar más si no está adyacente a su destino),  $h_2$  **domina** a  $h_1$ : es una heurística más informada –ambas son admisibles, pero  $h_2$  da estimados más ajustados y por tanto expande menos nodos.

#### 4.4. Heurística de Ignorar Listas de Borrado (Delete Lists)

**Definición 4.3** (Ignorar listas de borrado). Se asume que metas y precondiciones están compuestas **solo por literales positivos**, y se remueven las listas de borrado de las acciones (los literales negados de sus efectos): ningún fluente se elimina nunca. El problema de planeación relajado resultante es resoluble en **tiempo polinomial** mediante ascenso de colinas (*hill climbing*) sobre el espacio de estados, donde los arcos son acciones y la altura de cada punto es el valor heurístico.

##### 4.4.1. Grafo de Planeación y Heurística FF

Un **grafo de planeación** alterna capas de literales y acciones:

- Capa de literales  $S_i$ : literales alcanzables en  $i$  pasos.
- Capa de acciones  $A_i$ : acciones aplicables en el paso  $i$ .

Se construye en tiempo polinomial (a diferencia del espacio de estados completo) y la heurística **FF** (*Fast-Forward*) usa la longitud del plan hallado sobre el grafo relajado (ignorando listas de borrado) como estimador del costo restante.

#### 4.5. Podado Independiente de Dominio

Además de relajar el problema para obtener una heurística, se pueden podar ramas del árbol de búsqueda sin perder –o aceptando perder– la garantía de optimalidad.

**Reducción por simetría.** Se podan todas las ramas simétricas del árbol de búsqueda salvo una. Dos acciones son **simétricas** si generan dos estados equivalentes en el contexto del problema: mismas precondiciones y efectos estructurales, difiriendo solo en objetos que la meta no distingue entre sí.

**Ejemplo 4.3** (Mundo de bloques). Con estado inicial  $Sobre(A, Mesa) \wedge Sobre(B, Mesa) \wedge Sobre(C, Mesa) \wedge Despejar(A) \wedge Despejar(B) \wedge Despejar(C)$  y meta  $Sobre(A, B)$ , las acciones  $Mover(A, Mesa, B)$  y  $Mover(A, Mesa, C)$  tienen la misma estructura (mover  $A$  a la mesa hacia otro bloque despejado): como la meta no distingue entre  $B$  y  $C$ , ambas ramas son simétricas y basta explorar una.

**Verificación paso a paso de la simetría:**

1. **Mismas precondiciones:**  $Mover(A, Mesa, B)$  exige  $Despejar(B)$ ;  $Mover(A, Mesa, C)$  exige  $Despejar(C)$  –ambas se cumplen en el estado inicial, con la misma estructura (mover  $A$  hacia un bloque despejado cualquiera).
2. **Mismos efectos estructurales:** ambas acciones producen  $Sobre(A, \cdot)$  sobre un bloque que queda despejado antes de mover, difiriendo solo en si ese bloque se llama  $B$  o  $C$ .
3. **La meta no distingue:**  $Sobre(A, B)$  menciona a  $B$  explícitamente, pero nada en el problema obliga a preferir  $B$  sobre  $C$  antes de decidir –ambos objetos son intercambiables en este punto de la búsqueda.
4. Como las tres condiciones se cumplen, se declara la simetría y se **descarta** una de las dos ramas (p. ej.  $Mover(A, Mesa, C)$ ) sin perder la solución óptima, reduciendo a la mitad el factor de ramificación efectivo en este paso.

**Acciones preferidas.** Se asume el riesgo de podar una solución óptima a cambio de guiar la búsqueda rápidamente: se define una versión **relajada** del problema, se resuelve para obtener un **plan relajado**, y se llama **acción preferida** a cualquier acción que sea un paso de ese plan relajado o que logre alguna de sus precondiciones. La búsqueda hacia adelante explora primero las acciones preferidas.

**Ejemplo 4.4.** Con el mismo estado inicial y meta  $Sobre(A, B)$  del ejemplo anterior, las seis acciones posibles son  $Mover(A, Mesa, B)$ ,  $Mover(A, Mesa, C)$ ,  $Mover(B, Mesa, A)$ ,  $Mover(B, Mesa, C)$ ,  $Mover(C, Mesa, A)$ ,  $Mover(C, Mesa, B)$ . La relajación (ignorar precondiciones/borrado) resuelve la meta en un paso con  $Mover(A, Mesa, B)$ , así que esa es la **acción preferida**: se explora antes que las otras cinco.

**Paso a paso de la relajación:**

1. Construir el problema relajado: se ignoran las precondiciones (todas las 6 acciones quedan aplicables de inmediato) y las listas de borrado de sus efectos.
2. Revisar el efecto de cada una de las 6 acciones: solo  $Mover(A, Mesa, B)$  tiene  $Sobre(A, B)$  –el fuente exacto de la meta– en su lista de agregado.
3. El plan relajado óptimo es, entonces,  $[Mover(A, Mesa, B)]$ , de un solo paso.
4. Como  $Mover(A, Mesa, B)$  es un paso de ese plan relajado, se marca como **acción preferida** y la búsqueda hacia adelante la explora antes que las otras cinco.

**Remover interacciones negativas (submetas serializables).** Un problema tiene **submetas serializables** si existe un orden en el que el planificador puede lograrlas una por una **sin deshacer** ninguna submeta ya alcanzada (sin necesidad de backtracking).

**Ejemplo 4.5.** En el mundo de bloques, para construir una torre  $A$  sobre  $B$  sobre  $C$  sobre la mesa, lograr primero “ $C$  sobre la mesa” no requiere deshacerse después: las submetas son serializables en ese orden. Un contraejemplo clásico son  $n$  interruptores que controlan cada uno una luz separada, con meta “todas las luces encendidas”: si se define un orden particular de submetas, ninguna acción posterior interfiere con las anteriores.

**Paso a paso (torre  $A$  sobre  $B$  sobre  $C$ ):**

1. Lograr  $Sobre(C, Mesa)$ : ya es verdadero en el estado inicial (o se logra sin depender de  $A$  ni  $B$ ).
2. Lograr  $Sobre(B, C)$ : la acción  $Mover(B, Mesa, C)$  no toca a  $A$  ni deshace  $Sobre(C, Mesa)$  –sigue siendo verdadero después.
3. Lograr  $Sobre(A, B)$ : la acción  $Mover(A, Mesa, B)$  no deshace  $Sobre(B, C)$  ni  $Sobre(C, Mesa)$ .
4. Como ningún paso deshizo una submeta ya lograda, el orden  $C, B, A$  es una **serialización válida** –no hizo falta backtracking.

**Paso a paso ( $n$  interruptores):** con  $Prender(I_i)$  afectando únicamente a su luz  $L_i$ , lograr  $L_1$ , luego  $L_2, \dots$ , luego  $L_n$  en cualquier orden nunca deshace una luz ya encendida, porque cada interruptor es independiente de los demás –las  $n$  submetas son serializables en *cualquier* orden.

**Abstracción de estado.** Muchos problemas reales tienen  $10^{100}$  estados o más; relajar las *acciones* (como arriba) no reduce el número de *estados*, así que evaluar la heurística sigue siendo costoso. La **abstracción de estado** ataca esto **ignorando algunos fluentes** por completo, reduciendo drásticamente el tamaño del espacio de estados sobre el que se calcula la heurística.

**Ejemplo 4.6** (Carga aérea). Con 10 aeropuertos, 50 aviones y 200 piezas de cargo, el espacio de estados completo tiene del orden de  $10^{405}$  estados. Si se supone que todos los paquetes están en 5 aeropuertos y todos comparten destino, y se **ignoran los fluentes** salvo los que registran un avión y un paquete en cada uno de esos 5 aeropuertos, el espacio abstracto se reduce a apenas  $10^{11}$  estados.

**De dónde salen estas magnitudes (orden de magnitud, no cálculo exacto):**

1. **Espacio completo:** cada uno de los 200 cargos puede estar en cualquiera de  $\approx 60$  ubicaciones (10 aeropuertos o dentro de alguno de los 50 aviones), y cada uno de los 50 aviones puede estar en cualquiera de los 10 aeropuertos: el producto de todas estas posibilidades independientes ( $\approx 60^{200} \times 10^{50}$ ) ya está en el orden de  $10^{405}$ .
2. **Espacio abstracto:** si solo interesa, para cada uno de los 5 aeropuertos relevantes, si hay o no un avión presente y si hay o no un paquete presente (dos posibilidades cada uno), el número de combinaciones relevantes se reduce en muchísimos órdenes de magnitud –hasta quedar en el orden de  $10^{11}$ , todavía grande, pero  $\approx 10^{394}$  veces más pequeño que el espacio completo.
3. **Consecuencia:** calcular (o aproximar) una heurística sobre  $10^{11}$  estados abstractos es factible; hacerlo sobre  $10^{405}$  estados reales no lo es.

**Ideas clave para construir heurísticas: descomposición.** Se divide el problema en partes, se resuelve cada parte por separado, y se combinan los resultados, bajo la **suposición de independencia de submetas**: el costo de resolver una conjunción de submetas es, aproximadamente, la suma de los costos de resolver cada submeta por separado. Esta suposición puede fallar en dos direcciones:

- **Optimista:** existen interacciones negativas entre los subplanes de cada submeta (un subplan deshace el progreso de otro), así que el costo real es **mayor** que la suma.
- **Pesimista:** los subplanes contienen acciones redundantes (dos acciones se pueden reemplazar por una sola que sirva a ambas submetas), así que el costo real es **menor** que la suma.

**Costo de heurísticas por descomposición:** sea una meta con fluentes  $G$  dividida en submetas  $G_1, \dots, G_n$ , y sean  $P_1, \dots, P_n$  los planes óptimos que resuelven cada submeta por separado, con  $\text{COSTO}(P_i)$  como estimado heurístico de cada una.

- Combinar los estimados con el **máximo**,  $\max_i \text{COSTO}(P_i)$ , produce una heurística **admisible**.
- Combinar los estimados con la **suma**,  $\sum_i \text{COSTO}(P_i)$ , en general **no** es admisible (por las interacciones negativas de arriba).
- Es necesario escoger las abstracciones con cuidado, de modo que el costo de calcular la heurística sea menor que el costo de resolver el problema original –si no, la heurística no aporta nada.

**Ejemplo 4.7** (Máximo vs. suma). Sea  $G = G_1 \wedge G_2$ , con  $\text{COSTO}(P_1) = 3$  y  $\text{COSTO}(P_2) = 4$  resolviendo cada submeta por separado.

1. **Combinando con el máximo:**  $\max(3, 4) = 4$ . Esto nunca sobrestima el costo real, porque resolver *ambas* submetas juntas cuesta, como mínimo, lo que cuesta la más difícil de las dos por separado  $\Rightarrow$  admisible.
2. **Combinando con la suma:**  $3 + 4 = 7$ . Si existe una única acción que logra un fluente de  $G_1$  y un fluente de  $G_2$  a la vez (redundancia entre los subplanes), el costo real de lograr  $G_1 \wedge G_2$  juntas podría ser menor que 7 (p. ej. 5)  $\Rightarrow$  la suma **sobrestimaría** el costo real, violando la admisibilidad.

**Ejemplo 4.8** (Aplicación real: NASA Deep Space 1). La sonda *Deep Space 1* de la NASA usó restricciones temporales y de compatibilidad codificadas en un lenguaje similar a PDDL (*Remote Agent*) para planear y ejecutar maniobras autónomamente, ilustrando que estas técnicas de podado y relajación escalan a dominios reales de control de naves espaciales.

#### 4.6. Planeación Jerárquica (HTN)

Ni siquiera las heurísticas anteriores evitan que la planeación de bajo nivel sea inviable cuando la solución requiere **millones o miles de millones de acciones primitivas** (p. ej. “mover la rodilla 5 grados” repetido para cada micro-ajuste de un viaje completo). La **planeación jerárquica** (*Hierarchical Task Network*, HTN) ataca el problema organizando las acciones en **niveles de abstracción**, en vez de (o además de) usar heurísticas.

**Definición 4.4** (HTN). Una red jerárquica de tareas descompone **tareas abstractas** en **subtareas** más concretas, hasta llegar a **tareas primitivas** (acciones directamente ejecutables). Se asume observabilidad completa y determinismo.



- **Tareas primitivas:** acciones directamente ejecutables, con schema estándar (nombre, pre-condición, efecto).
- **Tareas abstractas (acciones de alto nivel, HLA):** se descomponen mediante **métodos de refinamiento** en secuencias de subtareas.
- **Ventaja:** permite incorporar conocimiento de dominio (qué refinamientos probar primero) para guiar la búsqueda, y –como se ve más adelante– reduce drásticamente el costo computacional frente a un planificador plano.

**Ejemplo 4.9** (Refinamiento recursivo). La acción de alto nivel  $Ir(Casa, SFO)$  admite (al menos) dos refinamientos:

REFINAMIENTO( $Ir(Casa, SFO)$ ),

PASOS : [ $Manejar(Casa, ParqueaderoSFO)$ ,  $Bus(ParqueaderoSFO, SFO)$ ]

REFINAMIENTO( $Ir(Casa, SFO)$ ), PASOS : [ $Taxi(Casa, SFO)$ ]

y una acción de alto nivel puede definirse **recursivamente**: por ejemplo,  $Navegar([a, b], [x, y])$  (ir de la celda  $(a, b)$  a la celda  $(x, y)$  en una cuadrícula) se refina en el caso base (ya se llegó) y en dos casos recursivos (moverse a la celda de la izquierda o de la derecha si está conectada):

REFINAMIENTO( $Navegar([a, b], [x, y])$ ), PRECOND :  $a=x \wedge b=y$ , PASOS : []

REFINAMIENTO( $Navegar([a, b], [x, y])$ ), PRECOND :  $Conectados([a, b], [a-1, b])$ ,

PASOS : [ $Izquierda$ ,  $Navegar([a-1, b], [x, y])$ ]

(y análogamente para  $Derecha$  con  $[a+1, b]$ ).

**Traza paso a paso de  $Navegar([0, 0], [2, 0])$** , suponiendo que  $[0, 0]$ – $[1, 0]$ – $[2, 0]$  están conectadas en línea recta:

1.  $Navegar([0, 0], [2, 0])$ :  $a=0$ ,  $b=0$ ,  $x=2$ ,  $y=0$ . Como  $a \neq x$ , no aplica el caso base; se cumple  $Conectados([0, 0], [1, 0])$ , así que se usa el refinamiento hacia la derecha: PASOS = [ $Derecha$ ,  $Navegar([1, 0], [2, 0])$ ].
2.  $Navegar([1, 0], [2, 0])$ : de nuevo  $a=1 \neq x=2$ ; se cumple  $Conectados([1, 0], [2, 0])$ : PASOS = [ $Derecha$ ,  $Navegar([2, 0], [2, 0])$ ].
3.  $Navegar([2, 0], [2, 0])$ : ahora  $a=x=2$  y  $b=y=0$ , se cumple la precondición del **caso base**: PASOS = [].
4. Concatenando las tres capas de refinamiento, el plan primitivo final es [ $Derecha$ ,  $Derecha$ ].

#### 4.6.1. Búsqueda para Soluciones Primitivas

**Formulación:** se define una acción de alto nivel raíz,  $Actuar$ , cuyo objetivo es encontrar una implementación que lleve a la meta. Para cada acción primitiva  $a_i$  del problema, se añade un refinamiento de  $Actuar$  con pasos  $[a_i, Actuar]$  (una definición recursiva que va añadiendo acciones); el ciclo se detiene añadiendo un refinamiento con una lista de pasos **vacía** cuya precondición es la meta del problema.

---

**Algorithm 2** HIERARCHICAL-SEARCH(*problem*, *hierarchy*) — búsqueda jerárquica en anchura

---

```

1: frontier  $\leftarrow$  cola FIFO con [Actuar] como único elemento
2: while verdadero do
3:   if frontier está vacía then return fracaso
4:   end if
5:   plan  $\leftarrow$  POP(frontier) ▷ el plan menos refinado de la frontera
6:   hla  $\leftarrow$  la primera HLA en plan (nulo si no hay ninguna)
7:   prefijo, sufijo  $\leftarrow$  subsecuencias de acciones antes/después de hla en plan
8:   resultado  $\leftarrow$  RESULTADO(problem.INICIAL, prefijo)
9:   if hla es nulo then ▷ el plan es primitivo; resultado es su efecto
10:    if problem.ESMETA(resultado) then return plan
11:    end if
12:   else
13:     for cada secuencia en REFINAMIENTOS(hla, resultado, hierarchy) do
14:       agregar CONCATENAR(prefijo, secuencia, sufijo) a frontier
15:     end for
16:   end if
17: end while

```

---

**Traza paso a paso** (meta: llegar a SFO, a partir de  $Ir(Casa, SFO)$  con los dos refinamientos del Ejemplo 4.8):

1.  $frontier = [Ir(Casa, SFO)]$ .
2. Se extrae  $[Ir(Casa, SFO)]$ :  $hla = Ir(Casa, SFO)$  (no nulo);  $prefijo = [], sufijo = []$ ;  $resultado = RESULTADO(Casa, []) = Casa$ .
3. Como *hla* no es nulo, se buscan sus refinamientos: REFINAMIENTOS( $Ir(Casa, SFO)$ , *Casa*, *hierarchy*) da las dos secuencias  $[Manejar(Casa, ParqueaderoSFO), Bus(ParqueaderoSFO, SFO)]$  y  $[Taxi(Casa, SFO)]$ , y ambas se agregan a la frontera:  $frontier = [[Manejar, Bus], [Taxi]]$  (en notación abreviada).
4. Se extrae el primer plan,  $[Manejar, Bus]$  (orden FIFO): ya no contiene ninguna HLA ( $hla = \text{nulo}$ ), así que es **primitivo**;  $resultado = RESULTADO(Casa, [Manejar, Bus]) = EstarEn(SFO)$ .
5. Como *problem*.ESMETA( $EstarEn(SFO)$ ) es verdadero, el algoritmo **retorna** el plan

$$[Manejar(Casa, ParqueaderoSFO), Bus(ParqueaderoSFO, SFO)]$$

—sin necesidad de examinar la rama  $[Taxi(Casa, SFO)]$ , que queda sin explorar en la frontera.

**Propiedades:** sea  $d$  el número de acciones primitivas de la solución y  $b$  el número de acciones aplicables por estado. Un planificador **no jerárquico** tiene costo  $\mathcal{O}(b^d)$ . Un planificador **jerárquico** donde cada acción no primitiva admite  $r$  refinamientos, cada uno con  $k$  acciones en el siguiente nivel, puede construir hasta  $r^{(d-1)/(k-1)}$  árboles de descomposición distintos —mucho menor que  $b^d$  cuando los refinamientos capturan bien la estructura del dominio. Además, el conocimiento de dominio también puede introducirse directamente en las precondiciones (PRECOND) de los refinamientos, y la estructura jerárquica resultante es más fácil de interpretar para humanos.

**Ejemplo numérico:** con  $b = 10$  acciones aplicables por estado y una solución de  $d = 20$  pasos primitivos, un planificador **no jerárquico** enfrenta hasta  $10^{20}$  nodos posibles. Si la jerarquía

ofrece  $r = 3$  refinamientos alternativos por HLA, cada uno con  $k = 4$  acciones en el siguiente nivel, el planificador **jerárquico** construye a lo sumo  $3^{(20-1)/(4-1)} = 3^{19/3} \approx 3^{6.3}$ , del orden de **unos pocos miles** de árboles de descomposición –muchísimos órdenes de magnitud menos que  $10^{20}$ .

#### 4.6.2. Búsqueda para Soluciones Abstractas: Semántica Angelical

La búsqueda anterior solo reconoce una solución cuando el plan ya es **completamente primitivo**, lo cual contradice el sentido común: debería ser posible afirmar que el plan

$$[\text{Manejar}(\text{Casa}, \text{ParqueaderoSFO}), \text{Bus}(\text{ParqueaderoSFO}, \text{SFO})]$$

llega al aeropuerto sin refinarlo hasta el nivel de presionar pedales. La solución es escribir descripciones de **precondición-efecto para las acciones de alto nivel**, igual que para las primitivas.

**Definición 4.5** (No determinismo demoníco vs. angelical). Cuando una acción de alto nivel tiene múltiples implementaciones posibles, hay dos formas de interpretar esa variabilidad:

- **Demónico**: un adversario escoge la implementación (como en juegos adversarios).
- **Angelical**: el **agente** escoge la implementación que más le convenga –entre más refinamientos tenga una acción de alto nivel, más poderosa es bajo esta semántica.

**Definición 4.6** (Set alcanzable, REACH). Dado un estado  $s$ , el **set alcanzable** de una acción de alto nivel  $h$ , escrito  $\text{REACH}(s, h)$ , es el conjunto de todos los estados alcanzables por *alguna* implementación primitiva de  $h$ . El set alcanzable de una secuencia se compone:

$$\text{REACH}(s, [h_1, h_2]) = \bigcup_{s' \in \text{REACH}(s, h_1)} \text{REACH}(s', h_2).$$

Un plan de alto nivel llega a la meta exactamente cuando su set alcanzable **interseca** el conjunto de estados meta.

**Notación de efectos posiblemente inciertos** (para acciones de alto nivel cuya implementación exacta aún no se fija):

- $\tilde{+}A$ : posiblemente añade  $A$ .
- $\tilde{-}A$ : posiblemente remueve  $A$ .
- $\tilde{\pm}A$ : posiblemente añade o remueve  $A$ .

**Ejemplo 4.10.** Si  $\text{Ir}(\text{Casa}, \text{SFO})$  tiene dos refinamientos (manejar+bus, o taxi) y el agente solo toma taxi si tiene dinero, entonces la acción de alto nivel *posiblemente* remueve *Dinero* (efecto  $\tilde{-}\text{Dinero}$ ). Con schemas abstractos para  $h_1$  y  $h_2$ :

$$\text{Acción}(h_1), \quad \text{PRECOND} : \neg A, \quad \text{EFECTO} : A \wedge \tilde{-}B$$

$$\text{Acción}(h_2), \quad \text{PRECOND} : \neg B, \quad \text{EFECTO} : \tilde{+}A \wedge \tilde{\pm}C$$

Como puede haber **infinitas** implementaciones distintas de una acción de alto nivel, se usan dos aproximaciones del set alcanzable real:

$$\text{REACH}^-(s, h) \subseteq \text{REACH}(s, h) \subseteq \text{REACH}^+(s, h),$$

donde  $\text{REACH}^+$  (descripción **optimista**) **sobreaproxima** el set alcanzable real, y  $\text{REACH}^-$  (descripción **pesimista**) lo **subaproxima**.

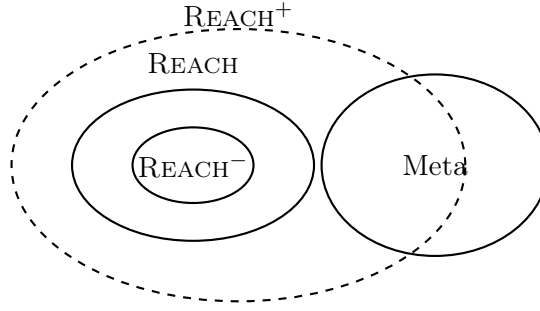


Figura 9:  $REACH^- \subseteq REACH \subseteq REACH^+$ : si  $REACH^-$  ya interseca la meta, hay solución garantizada; si ni siquiera  $REACH^+$  la interseca, no hay solución; si solo  $REACH^+$  la interseca, el resultado es ambiguo y hace falta refinar más.

### Verificación:

- Si  $REACH^+(s, plan)$  **no** interseca el set de metas  $\Rightarrow$  no existe solución por ese plan (ni ninguna de sus implementaciones puede llegar a la meta).
- Si  $REACH^-(s, plan)$  **sí** interseca el set de metas  $\Rightarrow$  existe solución garantizada (al menos una implementación llega a la meta).
- Si solo  $REACH^+$  interseca (y  $REACH^-$  no)  $\Rightarrow$  el resultado es **ambiguo**: hace falta refinar el plan más para decidir.

---

### Algorithm 3 ANGELIC-SEARCH(*problem*, *hierarchy*, *planInicial*)

---

```

1: frontier  $\leftarrow$  cola FIFO con planInicial como único elemento
2: while verdadero do
3:   if frontier está vacía then return fracaso
4:   end if
5:   plan  $\leftarrow$  POP(frontier)
6:   if  $REACH^+(problem.INICIAL, plan)$  interseca problem.META then
7:     if plan es primitivo then return plan
8:     end if
9:     garantizado  $\leftarrow REACH^-(problem.INICIAL, plan) \cap problem.META$ 
10:    if garantizado  $\neq \emptyset$  y HAYPROGRESO(plan, planInicial) then
11:      sf  $\leftarrow$  cualquier elemento de garantizado
12:      return DESCOMPONER(hierarchy, problem.INICIAL, plan, sf)
13:    end if
14:    hla  $\leftarrow$  alguna HLA en plan
15:    prefijo, sufijo  $\leftarrow$  subsecuencias antes/después de hla en plan
16:    resultado  $\leftarrow$  RESULTADO(problem.INICIAL, prefijo)
17:    for cada secuencia en REFINAMIENTOS(hla, resultado, hierarchy) do
18:      insertar CONCATENAR(prefijo, secuencia, sufijo) en frontier
19:    end for
20:  end if
21: end while

```

---

HAYPROGRESO verifica que la búsqueda no esté en un ciclo infinito de refinamientos; una vez se garantiza que *alguna* implementación llega a la meta, DESCOMPONER reconstruye la solución completa dividiendo el problema en subproblemas más pequeños, resolviendo cada uno de forma recursiva con ANGELICSEARCH.

**Traza paso a paso** (meta: estar en SFO,  $planInicial = [Ir(Casa, SFO)]$ , retomando el Ejemplo 4.8, donde *ambos* refinamientos de  $Ir(Casa, SFO)$  terminan en SFO):

1.  $frontier = [[Ir(Casa, SFO)]]$ .
2. Se extrae  $[Ir(Casa, SFO)]$ . Se calcula  $REACH^+(Casa, [Ir(Casa, SFO)])$ : la unión de los efectos de *ambos* refinamientos posibles (manejar+bus, o taxi) es  $\{SFO\}$ , que **interseca** la meta.
3. El plan  $[Ir(Casa, SFO)]$  **no** es primitivo (contiene una HLA), así que se calcula

$$REACH^-(Casa, [Ir(Casa, SFO)]) :$$

como los *dos* refinamientos posibles llegan a SFO sin excepción (no hay incertidumbre sobre el destino final),  $REACH^- = \{SFO\}$  también.

4.  $garantizado = REACH^- \cap META = \{SFO\} \neq \emptyset$ , y HAYPROGRESO confirma que no hay ciclo: se cumple la condición de la línea 10 del Algoritmo 3.
5. Se retorna  $DESCOMPONER(hierarchy, Casa, [Ir(Casa, SFO)], SFO)$ , que reconstruye una implementación primitiva concreta –por ejemplo,

$$[Manejar(Casa, ParqueaderoSFO), Bus(ParqueaderoSFO, SFO)]$$

– **sin** haber tenido que enumerar y comparar explícitamente ambos refinamientos acción por acción, como sí lo hacía HIERARCHICAL-SEARCH.

**Ejemplo 4.11** (Ventaja de la búsqueda angelical: mundo de aspiradora con  $D$  piezas). Sea un mundo con muchas piezas (habitaciones) conectadas por corredores, cada una de  $w \times h$  casillas, con la jerarquía “Navegar  $\rightarrow$  por pieza LimpiarTodaLaPieza  $\rightarrow$  limpiar cada fila”. Con  $D$  el tamaño de la solución mínima:

- BFS plano:  $\mathcal{O}(5^D)$  nodos.
- Búsqueda jerárquica (sin semántica angelical): mejora sobre BFS, pero sigue intentando encontrar **todas** las formas consistentes de limpiar cada pieza.
- Búsqueda angelical: escala **aproximadamente lineal** en el número de casillas, porque  $REACH^-/REACH^+$  permiten confirmar o descartar una acción de alto nivel completa sin enumerar sus implementaciones primitivas una por una.

**Ejemplo numérico:** sea un edificio de 5 piezas de  $3 \times 3$  casillas cada una, donde limpiar una pieza completa toma  $D_{pieza} \approx 9$  pasos primitivos (uno por casilla), para un total de  $D \approx 45$  pasos primitivos.

- BFS plano: hasta  $5^{45}$  nodos –completamente inviable.
- Búsqueda jerárquica (sin semántica angelical): reduce la búsqueda a nivel de *LimpiarTodaLaPieza* por pieza, pero sigue enumerando las distintas formas concretas de recorrer las 9 casillas de cada una.
- Búsqueda angelical: usando  $REACH^+/REACH^-$ , *LimpiarTodaLaPieza* se confirma o descarta como un solo paso de alto nivel –sin enumerar las 9 casillas internamente– así que el costo escala con el número de **piezas** (5), no con  $D$  (45).

## 5. Planeación en Ambientes No Deterministas

### 5.1. Ambientes No Deterministas y Condicionales

Cuando las acciones tienen efectos inciertos, se necesitan **planes condicionales** que incluyan ramificaciones según el resultado de las acciones.

- **Plan de búsqueda AND-OR:** los nodos AND representan contingencias del entorno; los nodos OR representan elecciones del agente.
- Un plan condicional es una solución si cubre todas las ramas.

### 5.2. Planeación Parcialmente Observable

Con observabilidad parcial el agente mantiene un **estado de creencia** (belief state) y ejecuta acciones de recopilación de información.

### 5.3. Planeación Continua y de Ejecución

- **Monitoreo de ejecución:** detectar discrepancias entre el plan y la realidad.
- **Replaneación:** adaptar el plan ante fallos o nueva información.
- **Agentes reactivos-deliberativos:** combinan planeación a largo plazo con respuesta reactiva.